

IUS
INSTITUT
UNIVERSITAIRE
DES SCIENCES

INSTITUT UNIVERSITAIRE DES SCIENCES

Université : (IUS)

FACULTÉ : Sciences Informatique (FST)

TITRE : TD n 1- Systèmes

NOM & PRENOM : Desvallons Olivier

Niveau : 3eme Année

Date : 18 Octobre 2026

Description des résultats de la tâche

1. Objectif du Travail Pratique (TD1)

L'objectif principal de ce travail pratique était de maîtriser les outils fondamentaux de contrôle de version (**Git**) et de plateforme d'hébergement distant (**GitHub**), tout en sécurisant les communications via le protocole **SSH**. Il s'agissait de mettre en place un environnement de travail local opérationnel sous Windows, d'effectuer le suivi de version de fichiers, d'associer une clé cryptographique à un compte GitHub, et enfin d'héberger le rapport final ainsi que l'ensemble des preuves d'exécution (captures d'écran) dans une arborescence structurée (TD/systeme/ et TD/reseaul/).

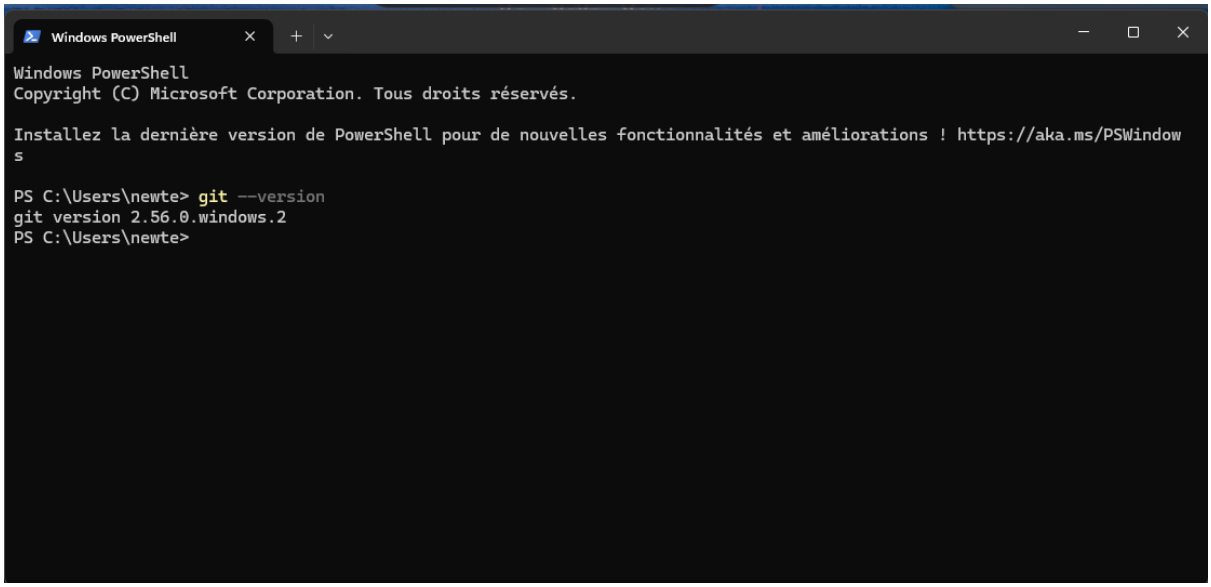
2. Démarche suivie

La réalisation de ce projet s'est articulée autour des grandes étapes méthodologiques suivantes :

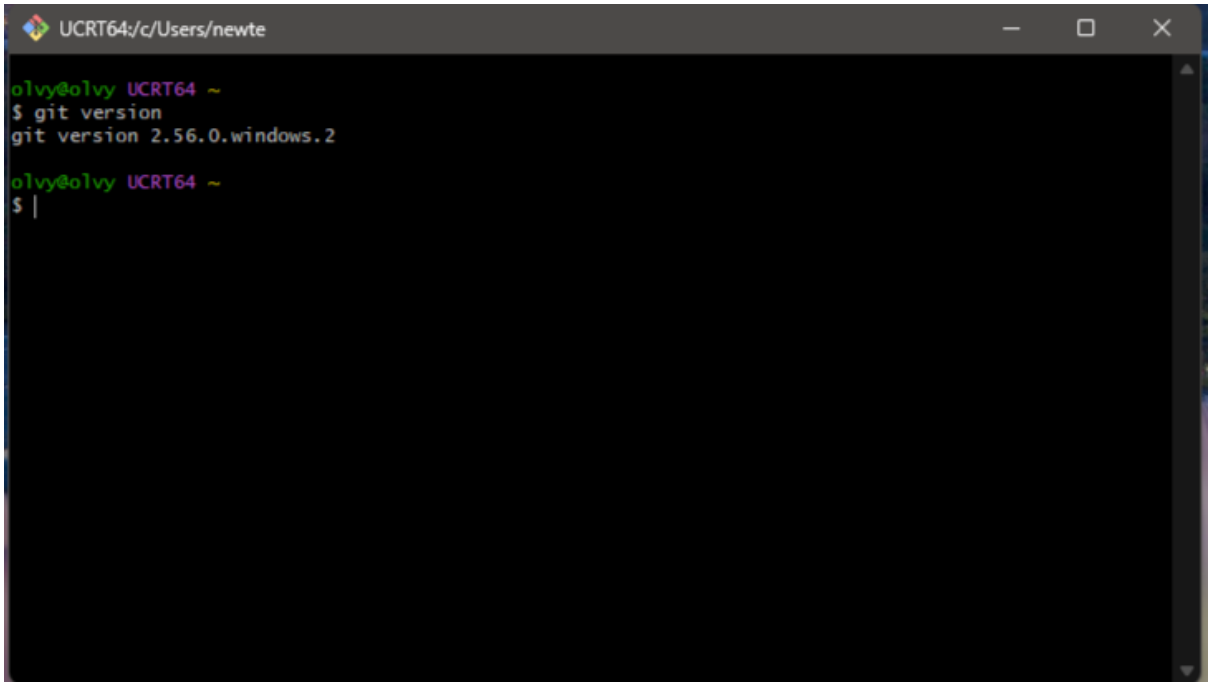
1. **Vérification et configuration initiale :**
    - Vérification de l'installation de Git via Windows PowerShell et Git Bash.
    - Définition des identifiants globaux de l'utilisateur (user.name et user.email).
2. **Gestion locale du dépôt de test :**
    - Création d'un dossier de travail (projet1-git) et initialisation d'un dépôt Git local (git init).
    - Création de fichiers (README.md, Module.txt, LICENSE), suivi de leur état (git status), indexation (git add), enregistrement des commits (git commit) et consultation de l'historique (git log).
3. **Sécurisation de la connexion SSH avec GitHub :**
    - Génération d'une paire de clés SSH RSA 4096 bits (ssh-keygen).
    - Lecture de la clé publique (id_rsa.pub) et enregistrement de celle-ci dans les paramètres de sécurité du compte GitHub.
4. **Synchronisation et interactions distantes :**
    - Création du dépôt distant sur GitHub et clonage en local via le lien SSH (git clone).
    - Publication et synchronisation des commits locaux vers la branche principale sur GitHub (git push -u origin main).
5. **Mise en place du dépôt final et organisation des livrables :**

- Création du dépôt principal nommé **TD** sur GitHub.
- Clonage du dépôt sur le bureau local et création de la structure hiérarchique de dossiers (systeme/image, systeme/projet, systeme/presentation, reseaul/...).
- Placement du rapport rédigé au format PDF (Rapport1.pdf) dans systeme/ et des captures d'écran dans systeme/image/, suivi de l'envoi final (*push*) sur le dépôt distant.

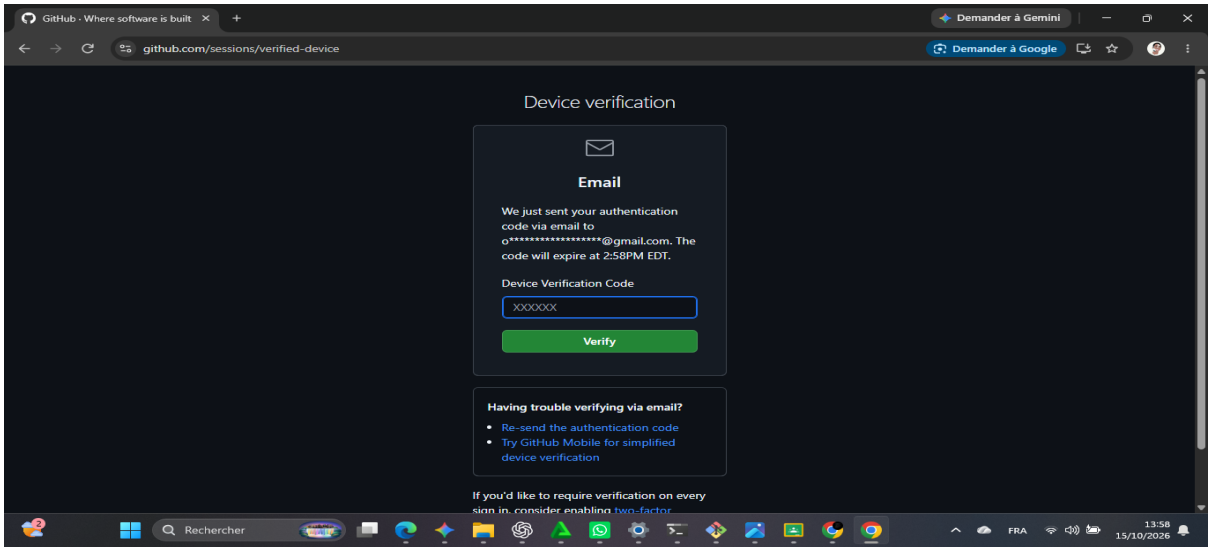
vérification de la version de Git installée sur le système à l'aide de la commande git --version dans Windows PowerShell. La version installée est la 2.56.0.windows.2.



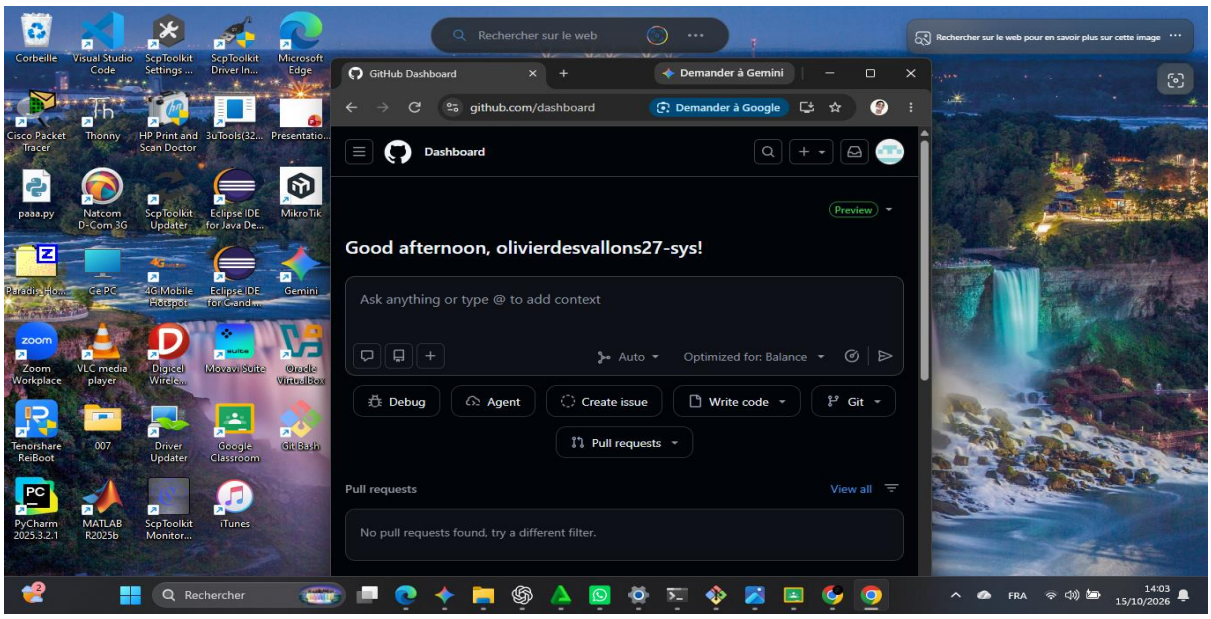
Vérification identique de la version de Git exécutée dans le terminal Git Bash avec la commande git version.



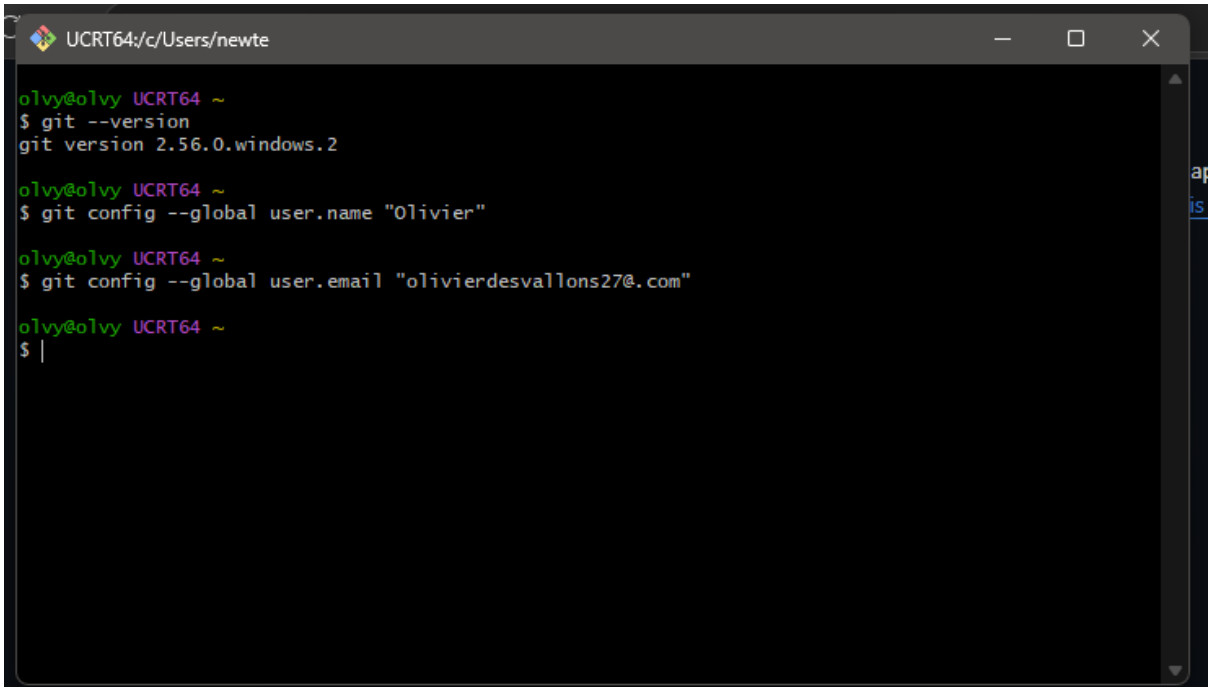
Écran de vérification de sécurité par e-mail (*Device verification*) lors de la connexion au compte GitHub depuis le navigateu



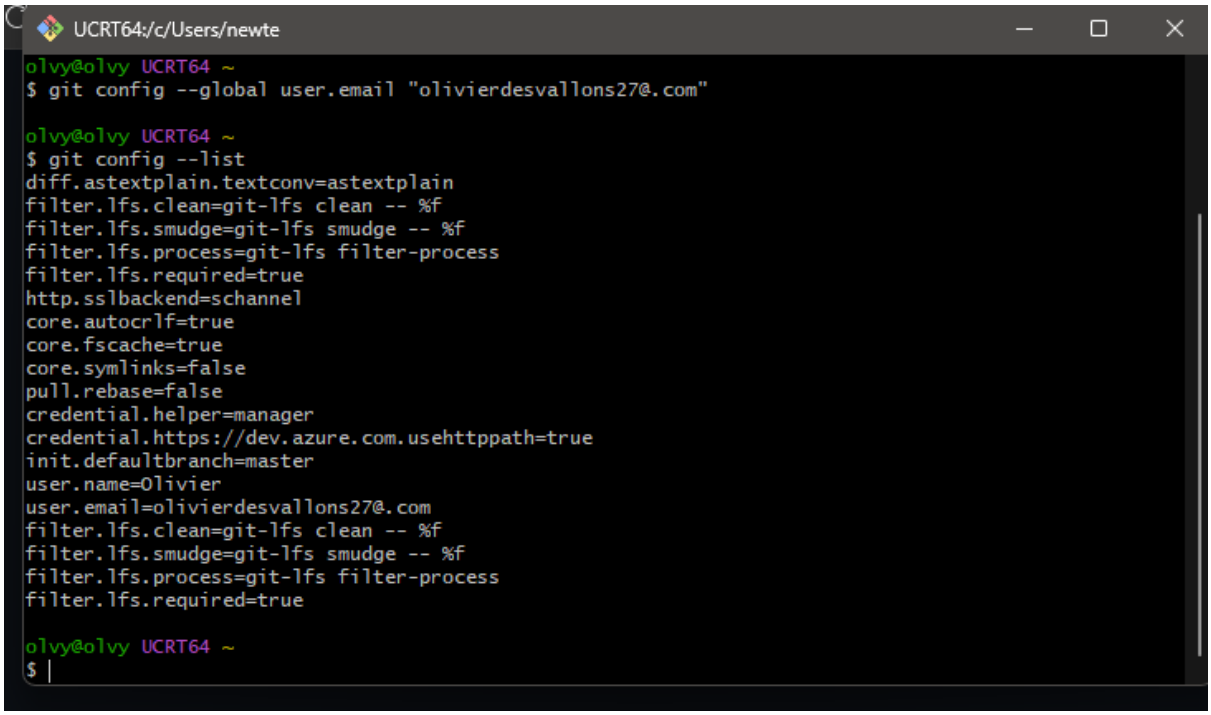
Accès réussi au tableau de bord (*Dashboard*) du compte GitHub sous l'identifiant olivierdesvallons27-sys.



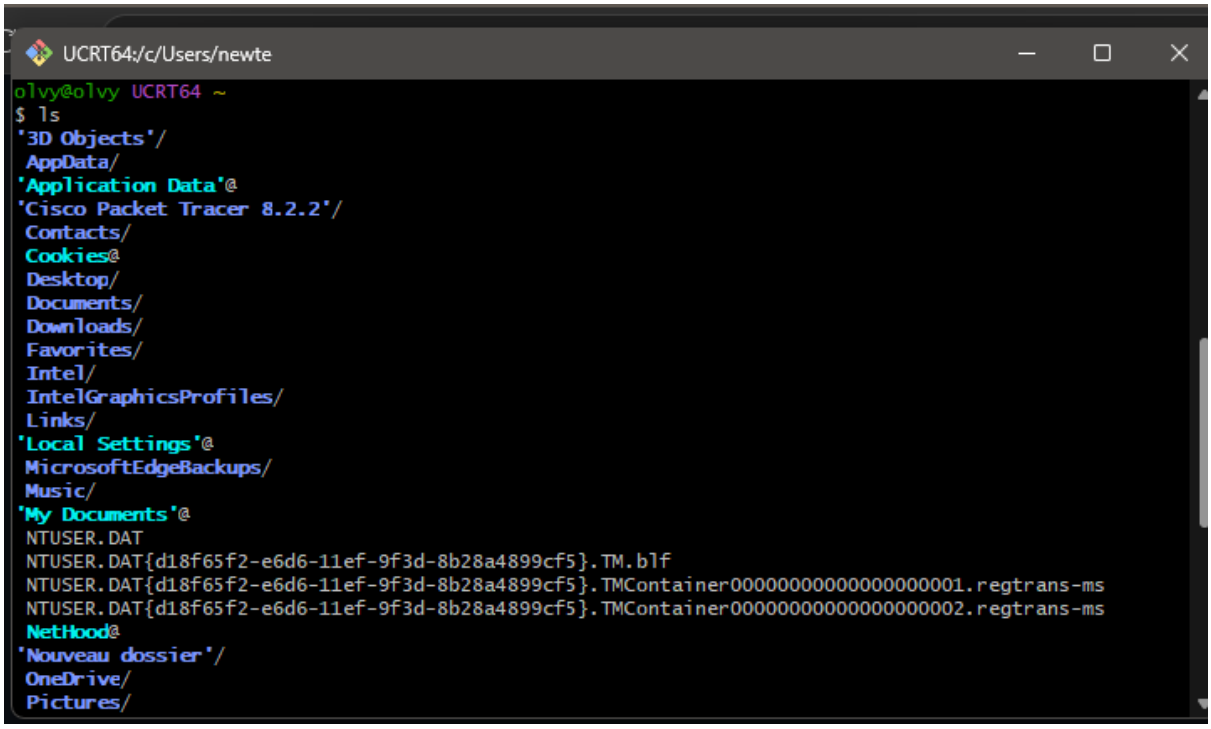
Configuration globale de l'identité Git avec les commandes git config --global user.name "Olivier" et git config --global user.email "olivierdesvallons27@gmail.com".



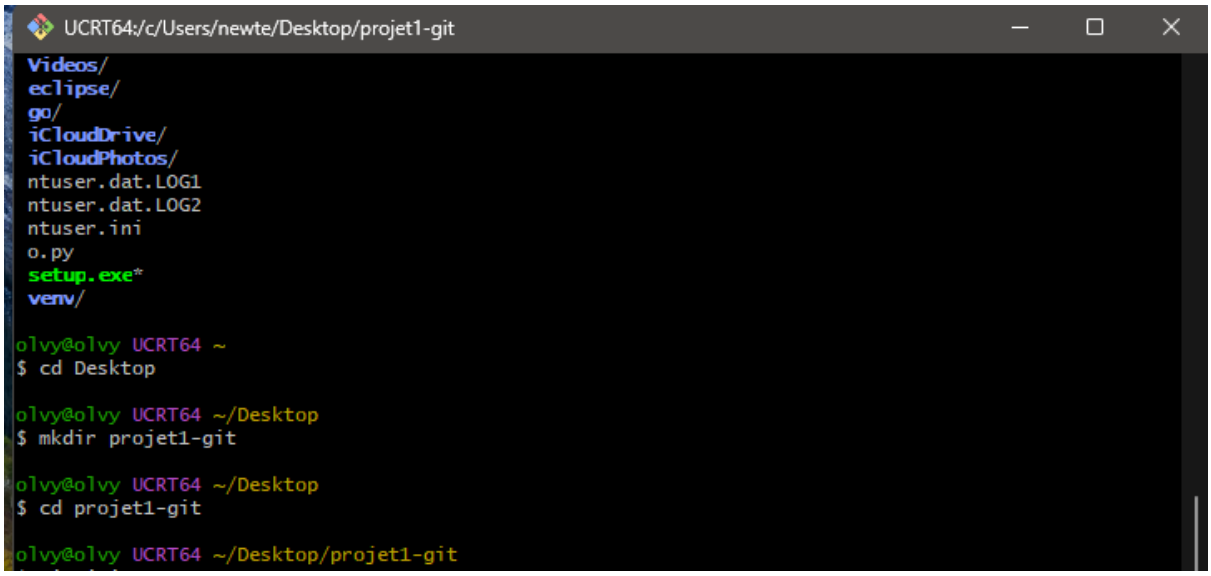
Affichage de toutes les configurations actives dans Git à l'aide de la commande git config --list.



: Utilisation de la commande ls pour lister le contenu du répertoire utilisateur personnel.



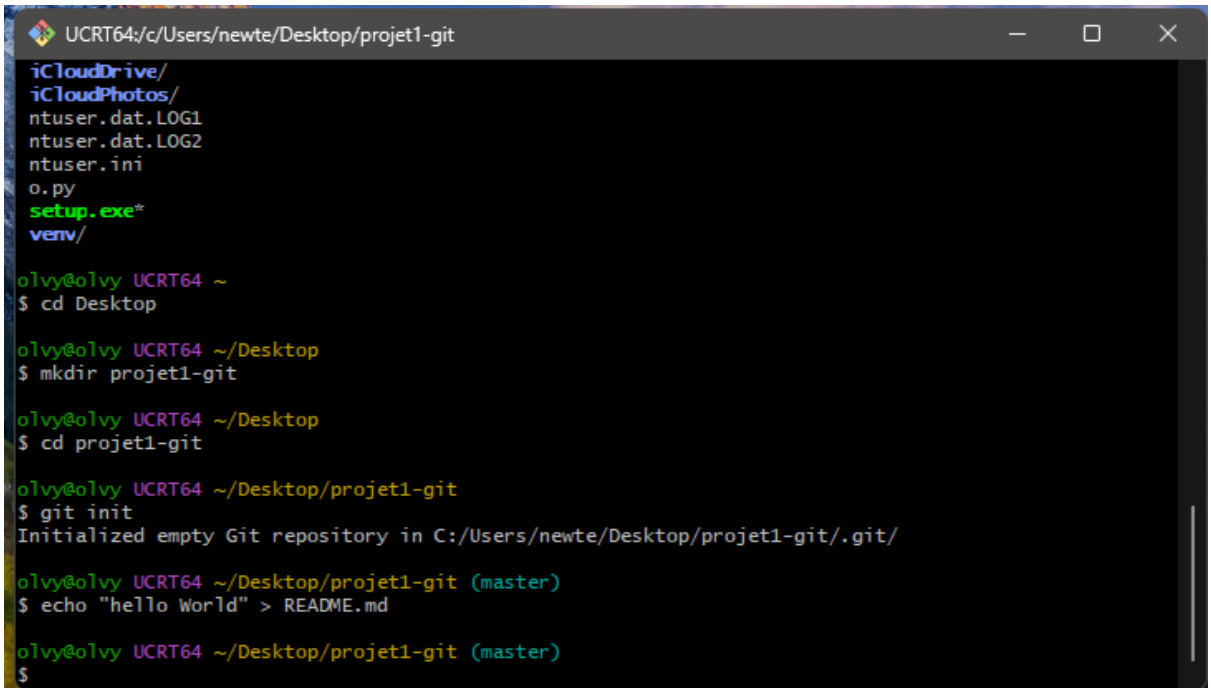
Navigation vers le Bureau (cd Desktop), création du dossier projet1-git (mkdir projet1-git), puis accès au dossier (cd projet1-git).



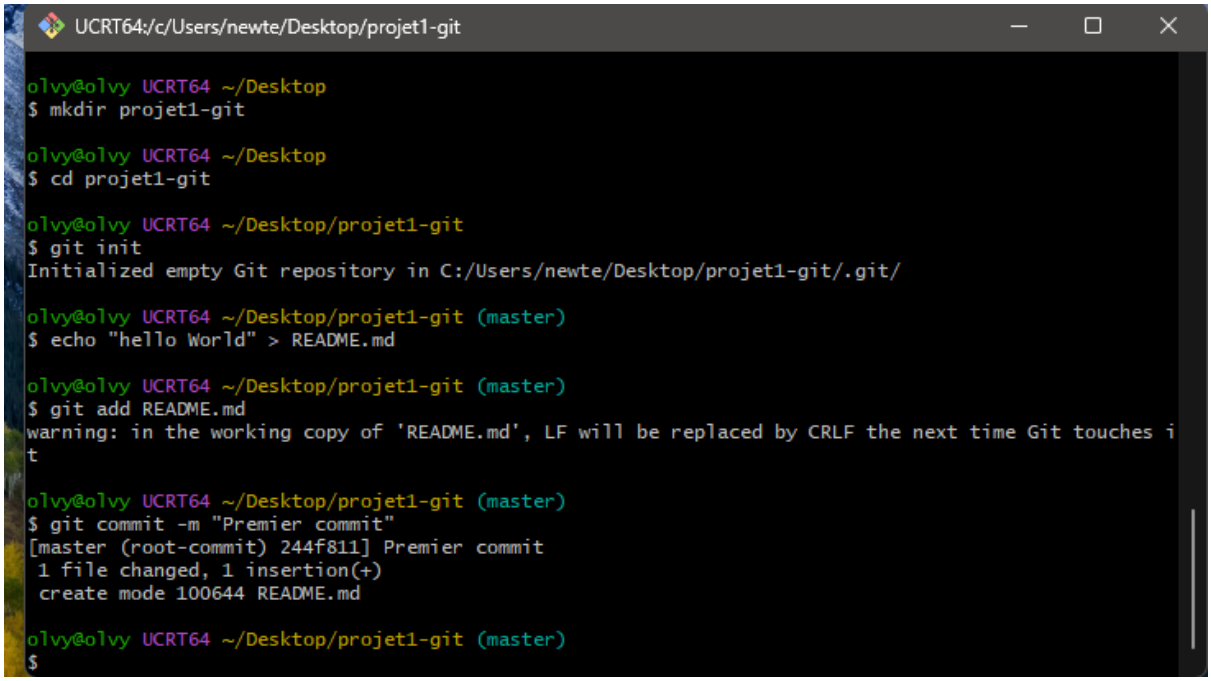
Icônes de projet1-git créés sur le bureau de l'ordinateur.



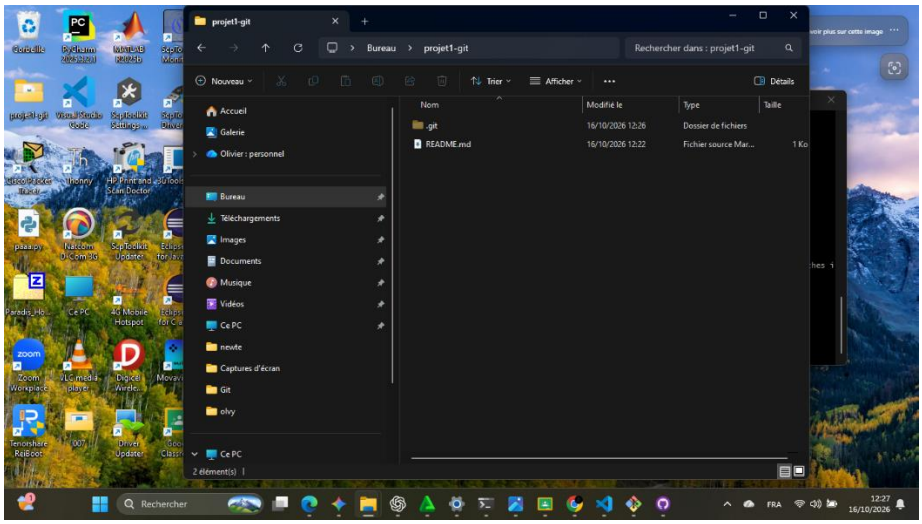
Initialisation du dépôt Git local avec git init et création du fichier README.md avec la commande echo "hello World" > README.md



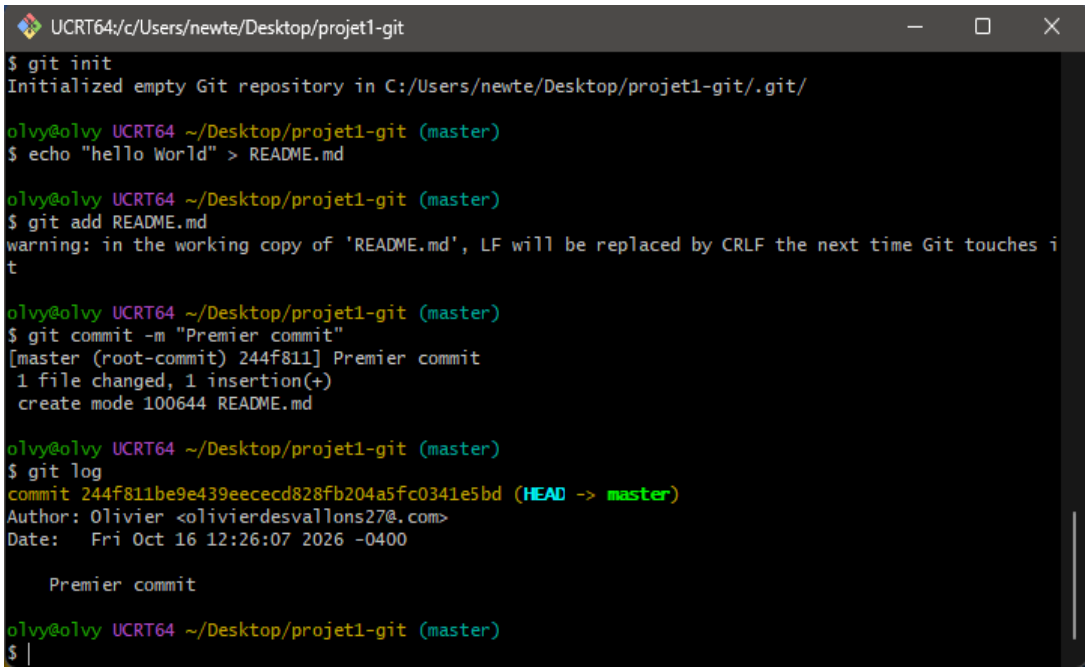
Ajout du fichier à la zone de transit (git add README.md) et enregistrement du premier commit (git commit -m "Premier commit")



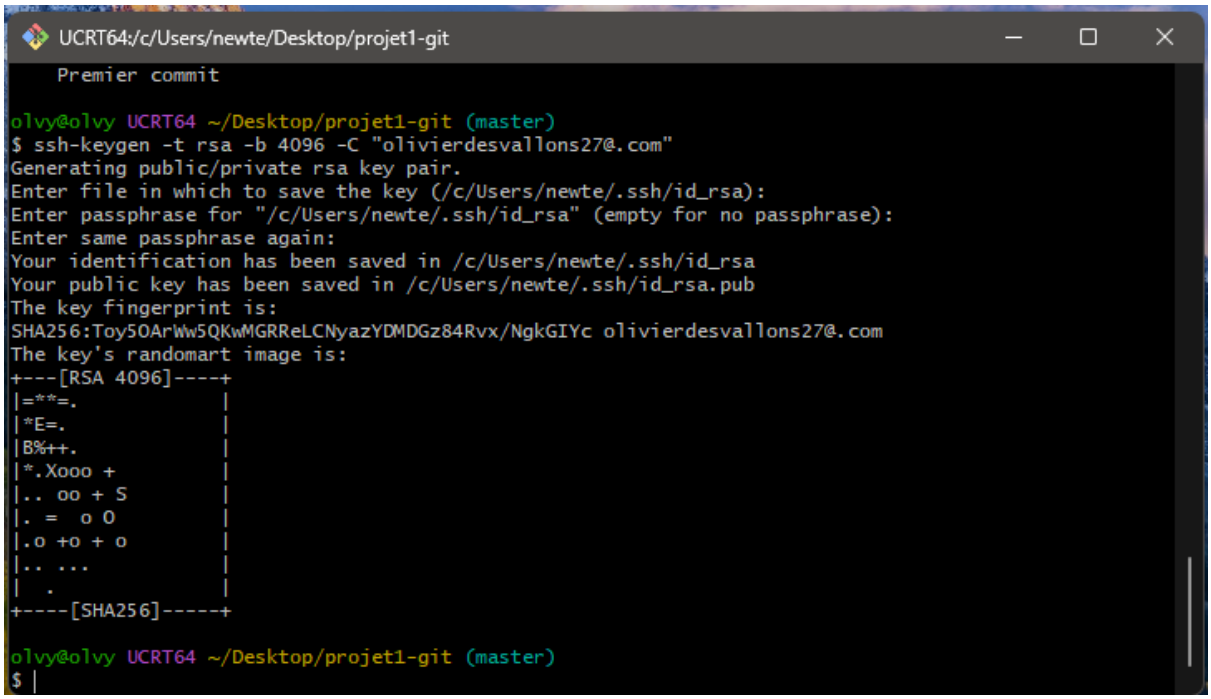
Aperçu dans l'Explorateur de fichiers montrant le fichier README.md présent à côté du dossier .git.



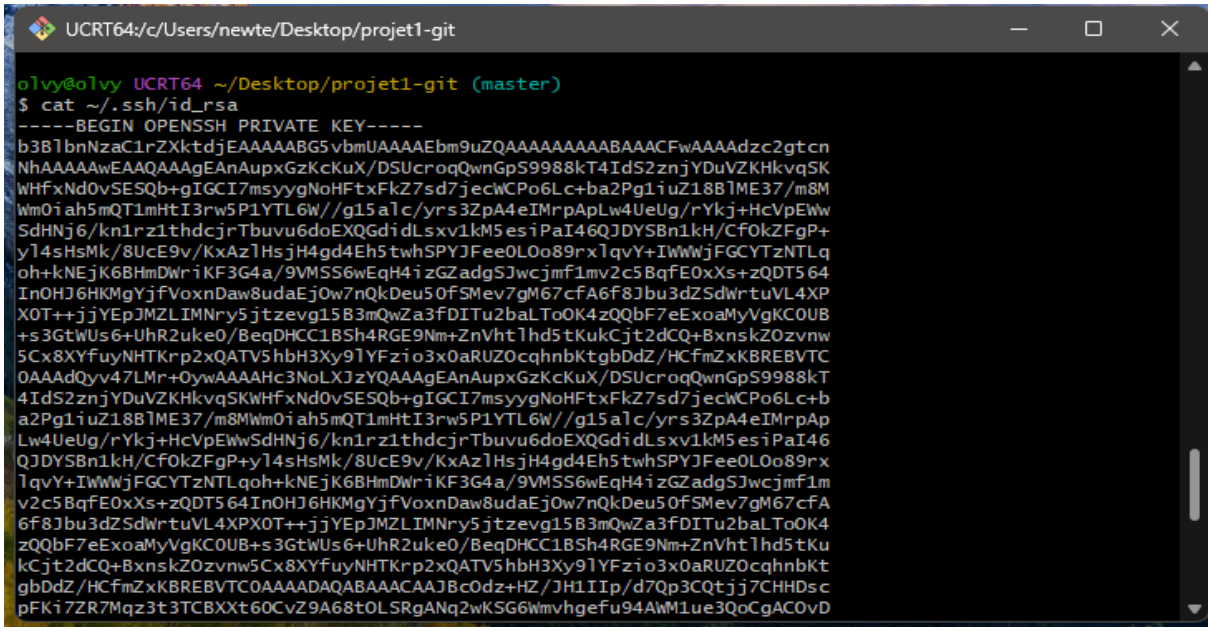
Affichage du journal des commits à l'aide de la commande git log, affichant le hash, l'auteur et la date.



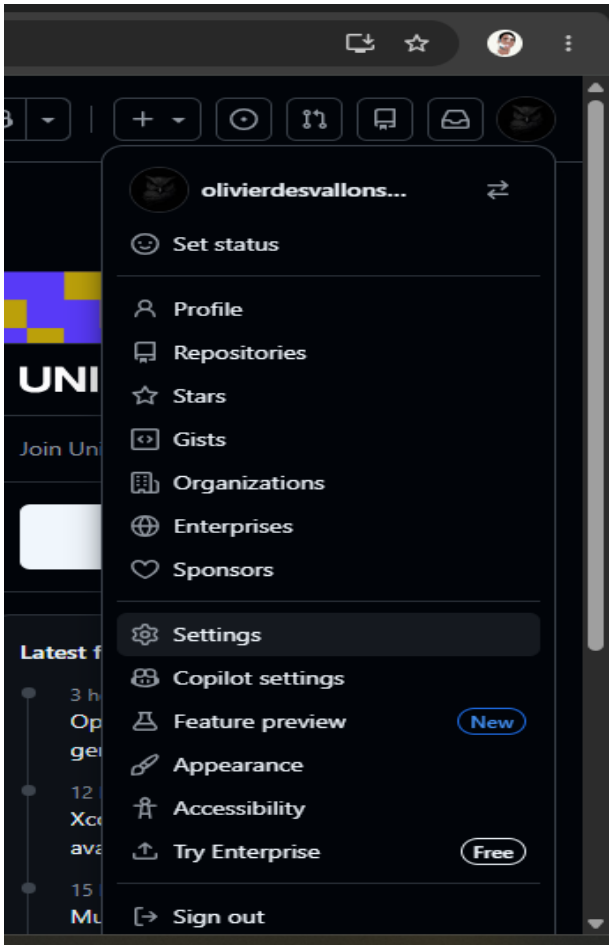
: Génération d'une clé SSH RSA 4096 bits associée à l'adresse e-mail via la commande ssh-keygen -t rsa -b 4096 -C "...".



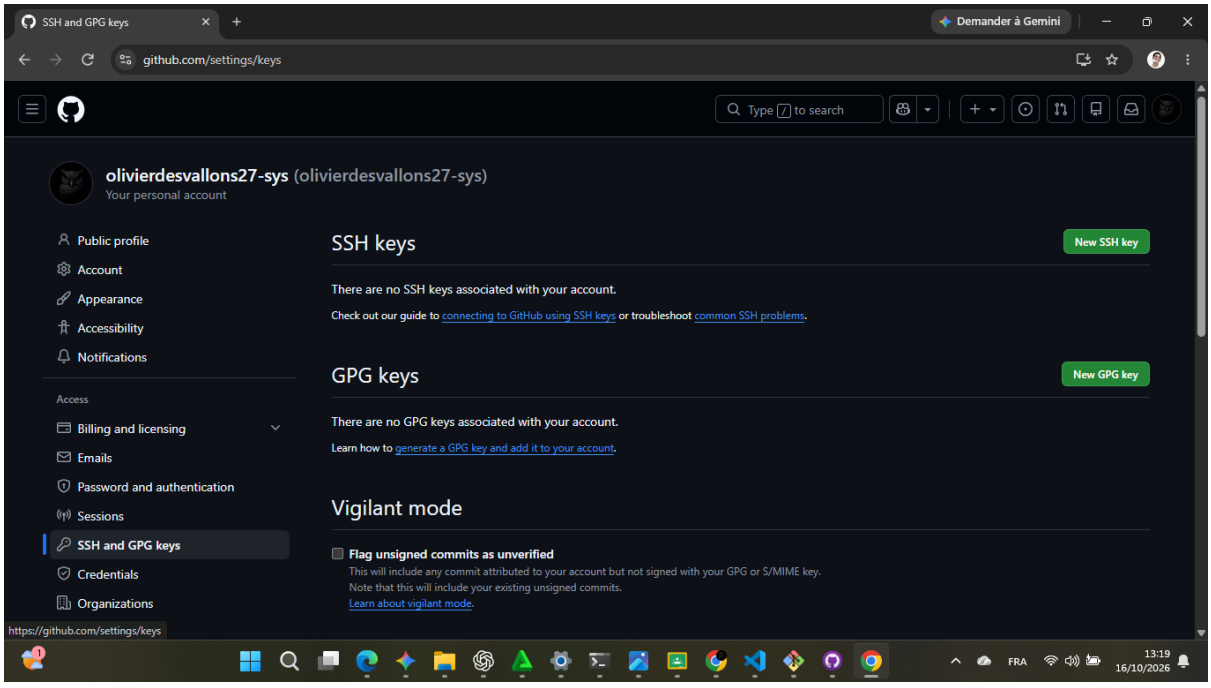
Affichage du contenu de la clé privée (cat ~/.ssh/id_rsa) dans le terminal.



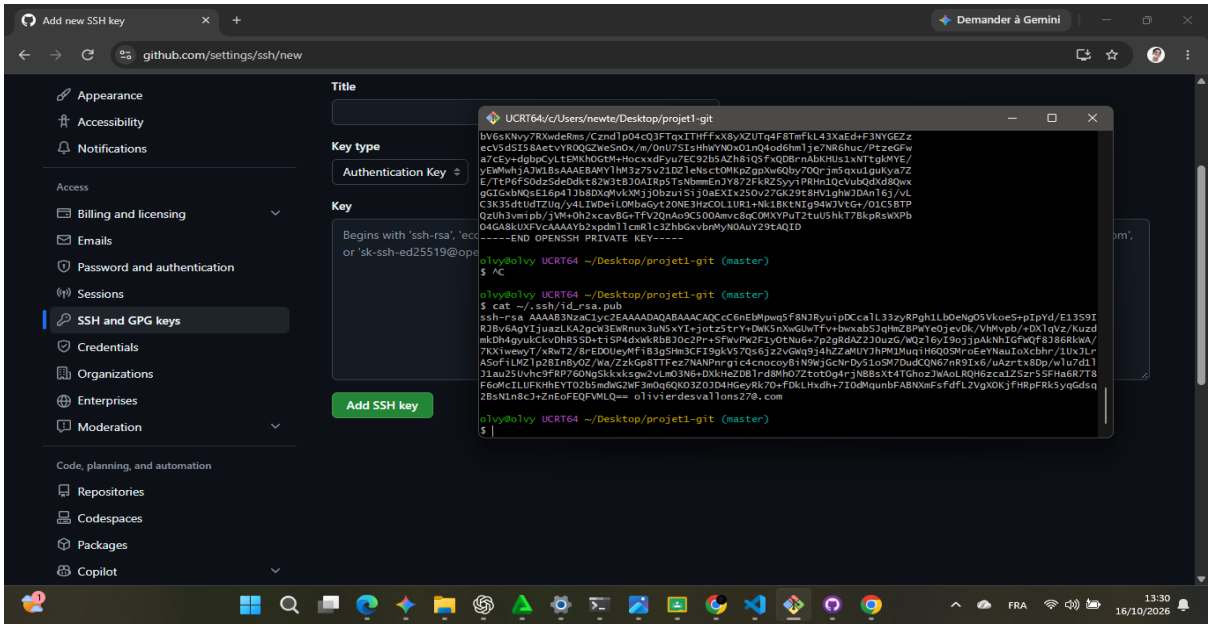
Navigation vers le menu des paramètres du compte (*Settings*) sur GitHub.



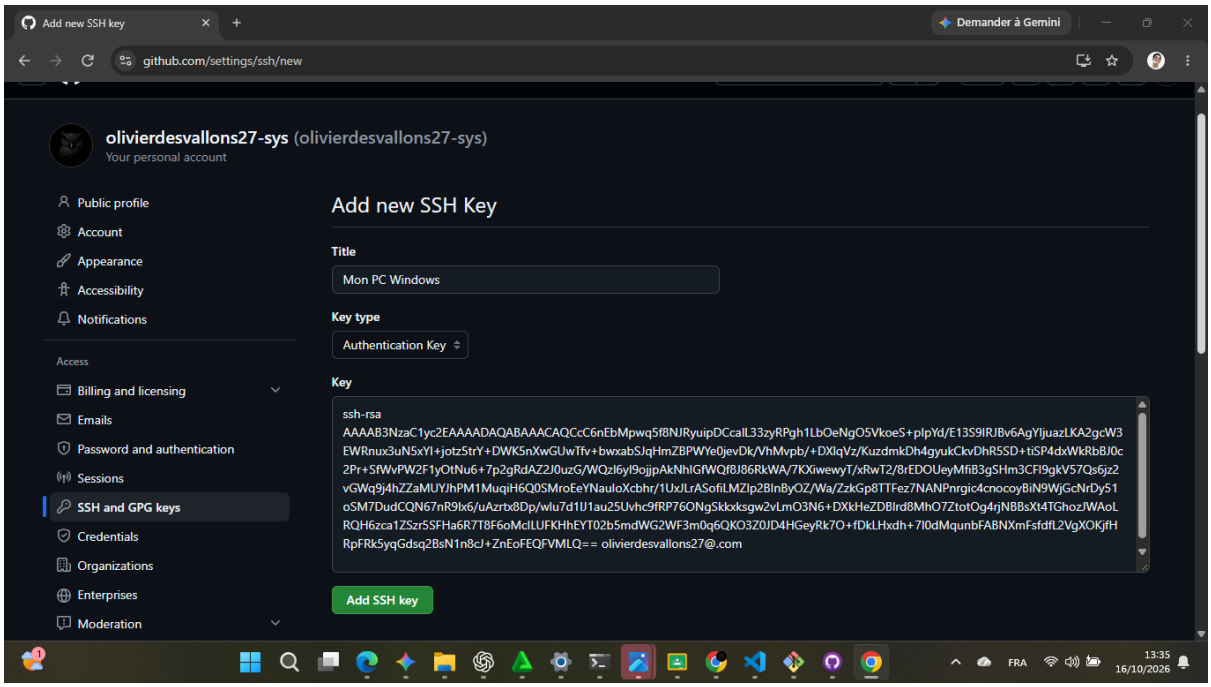
Accès à la sous-section **SSH and GPG keys** pour ajouter une nouvelle clé SSH.



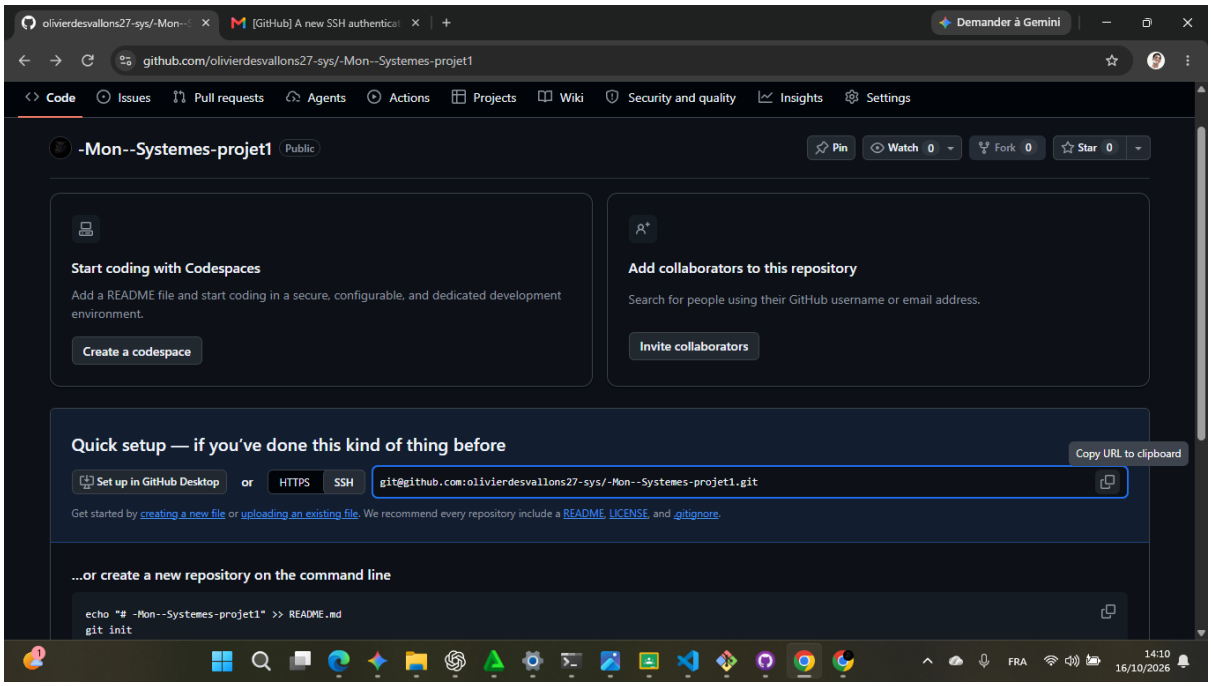
Affichage de la clé publique (cat ~/.ssh/id_rsa.pub) pour copie vers GitHub.



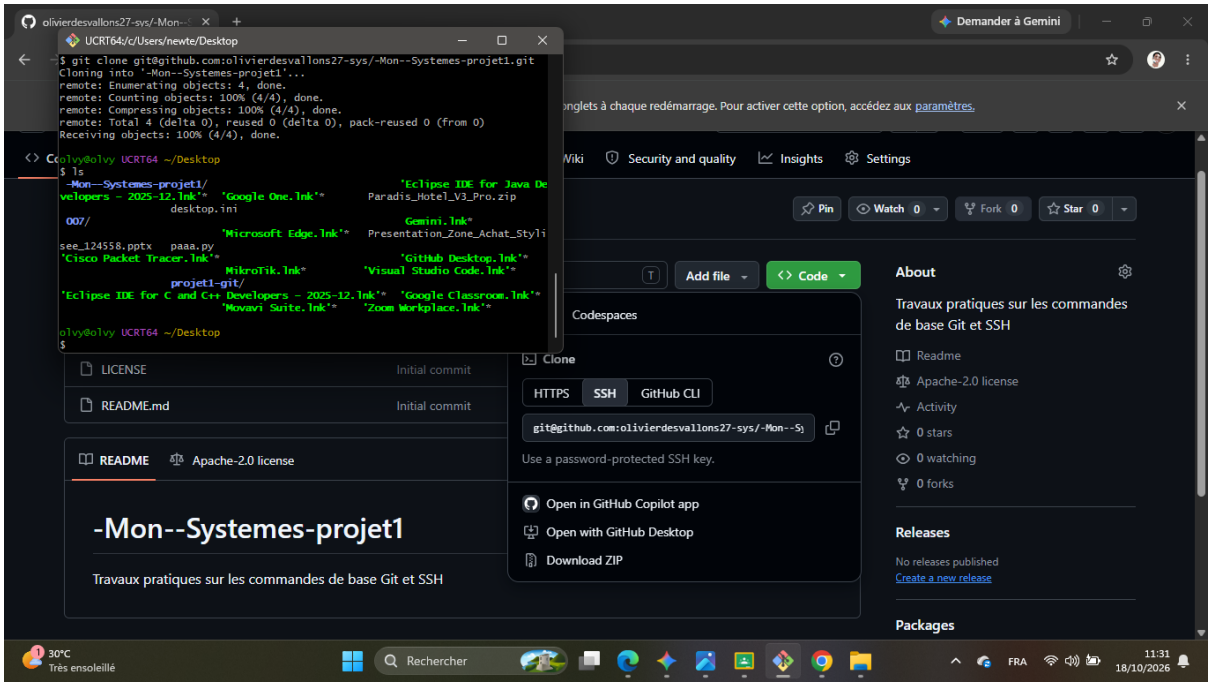
Collage de la clé publique SSH et attribution du nom Mon PC Windows avant validation sur GitHub.



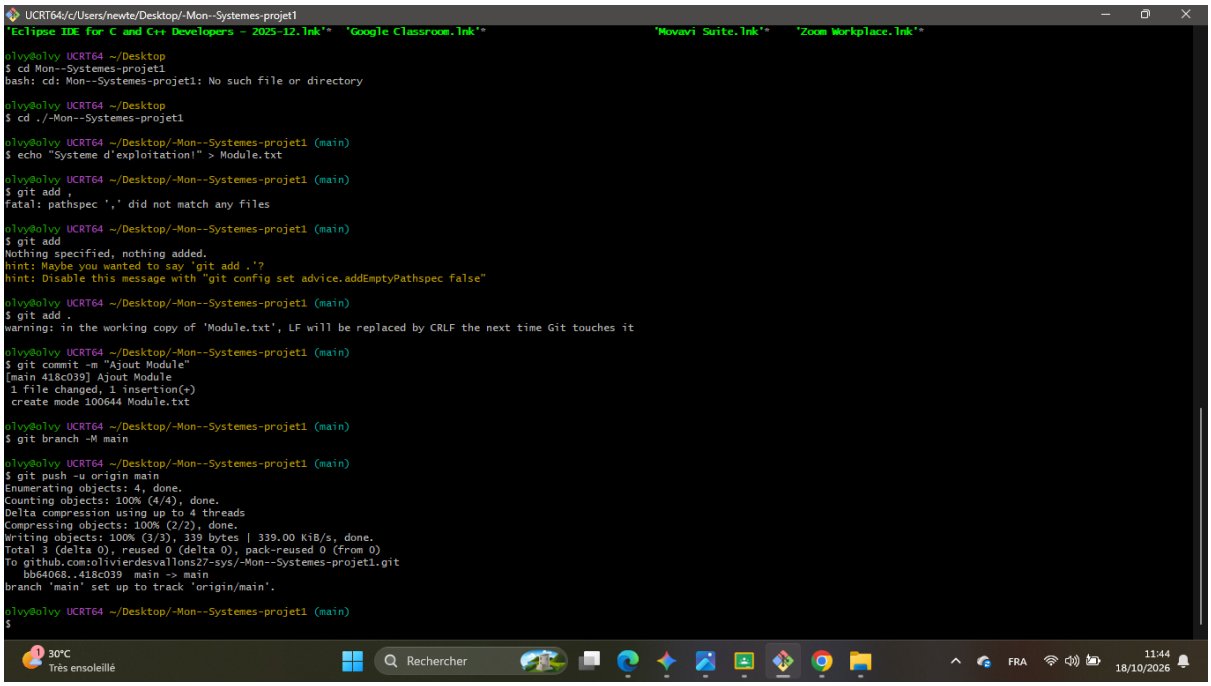
Récupération du lien SSH du dépôt distant pour pouvoir effectuer le clonage en local.



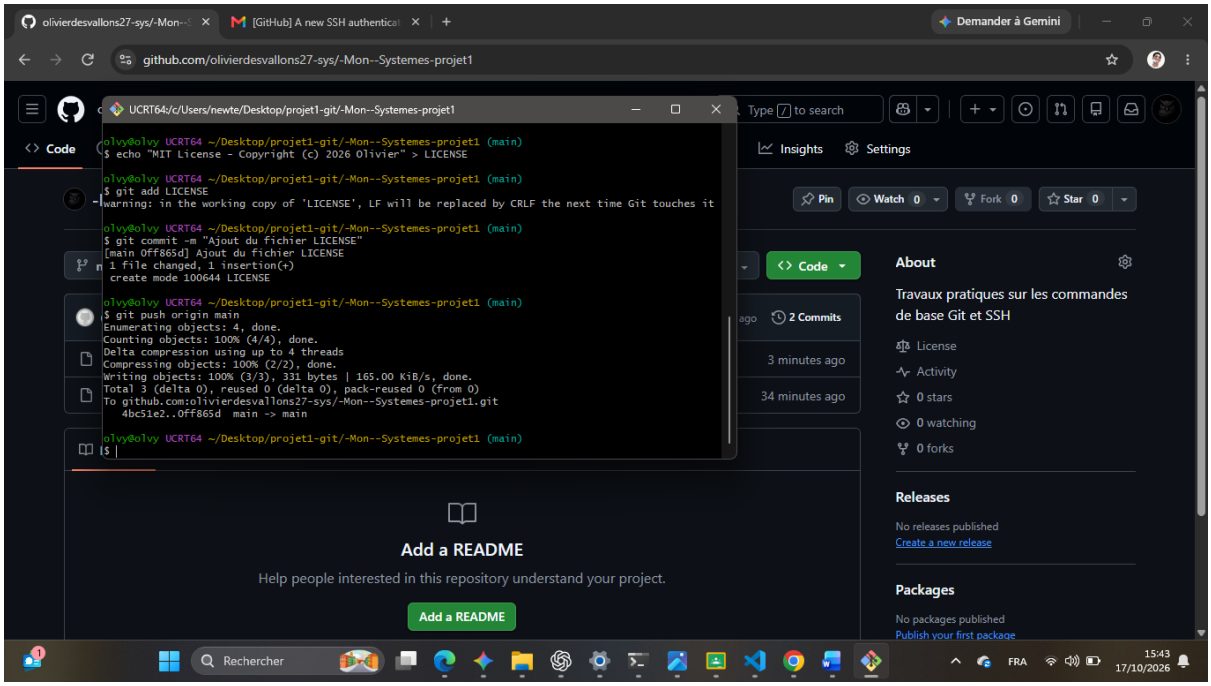
Execution de la commande git clone git@github.com:... et vérification du dossier cloné sur le Bureau avec ls



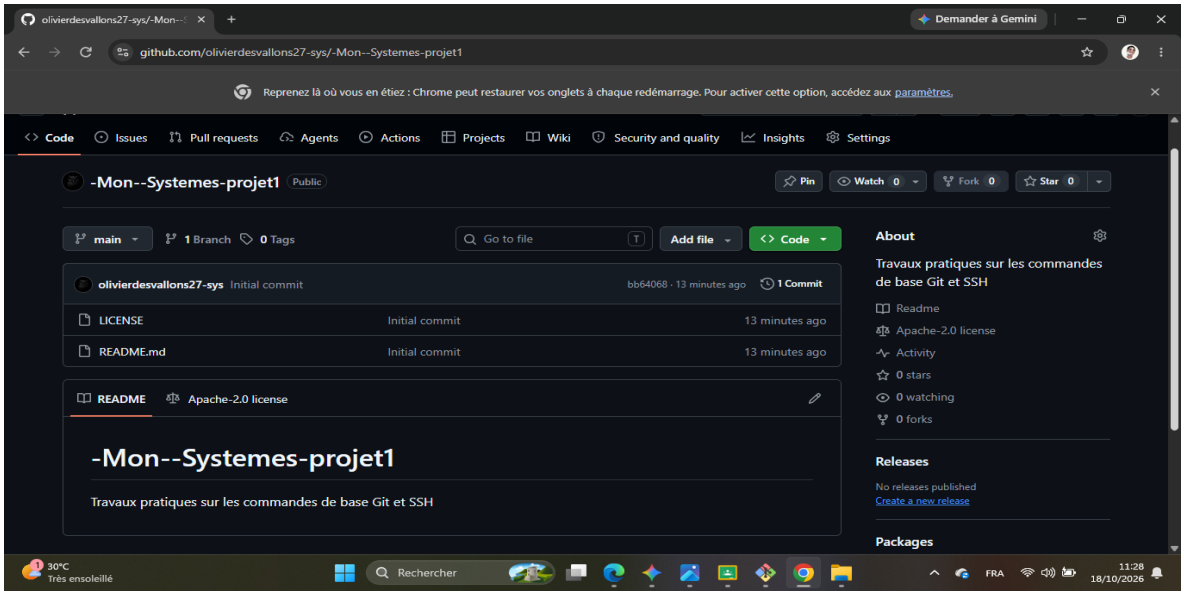
Création du fichier Module.txt, commit (git commit -m "Ajout Module") et envoi vers le serveur via git push -u origin main.



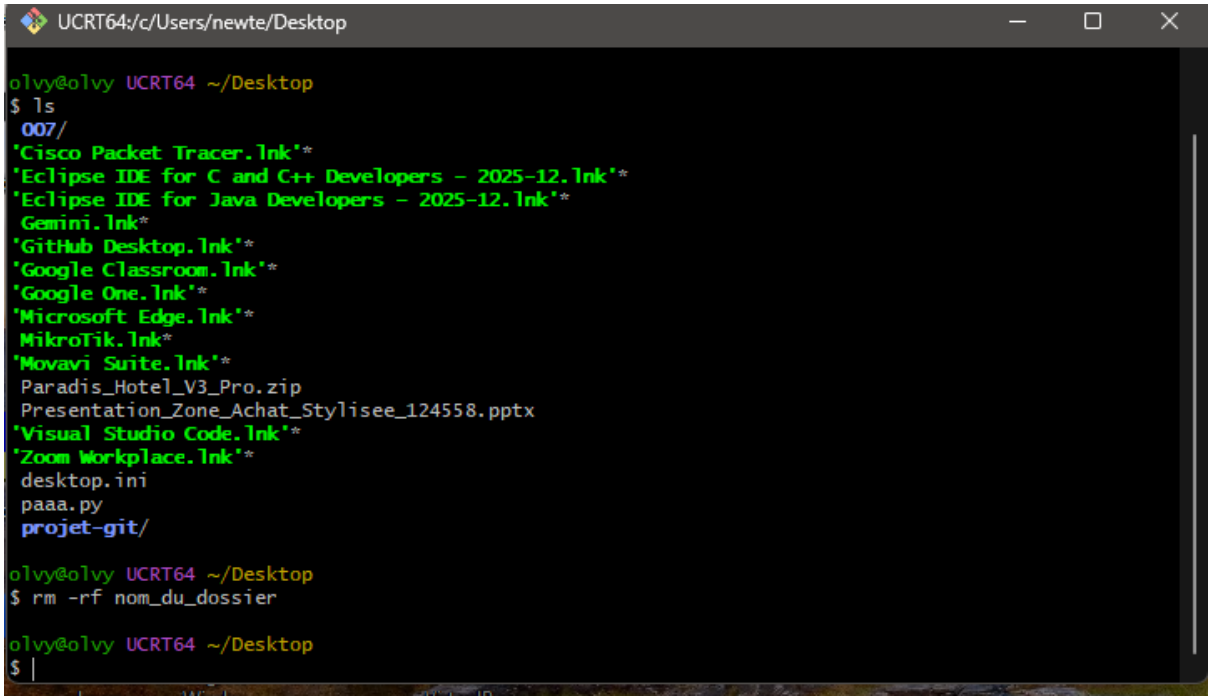
Création du fichier LICENSE et synchronisation réussie avec le dépôt distant via git push origin main.



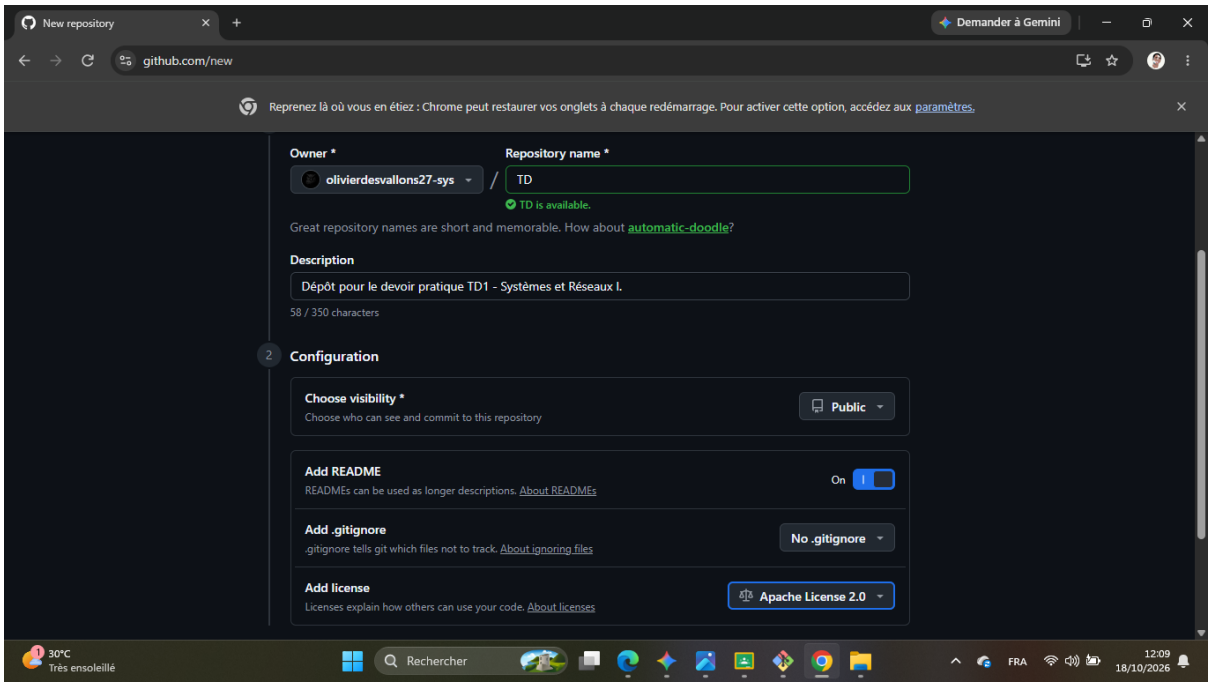
Consultation de l'interface web de GitHub confirmant la réception de tous les fichiers et de la clé SSH active



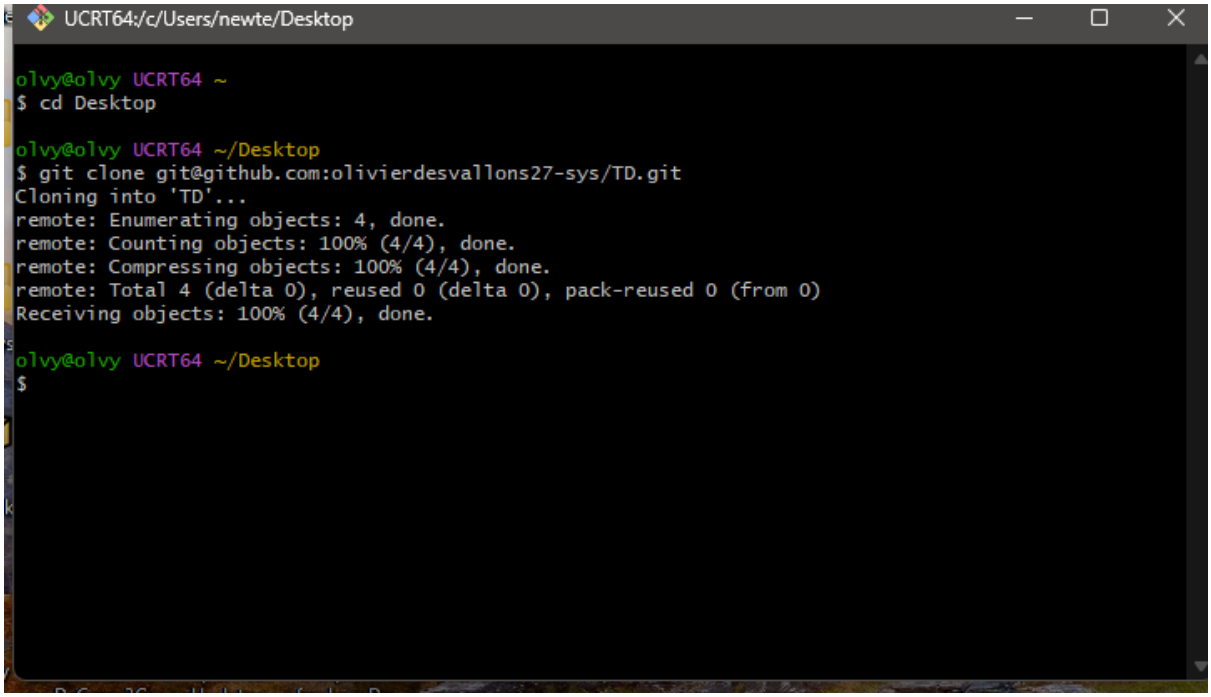
Suppression des dossiers temporaires inutiles sur le bureau à l'aide de la commande rm -rf



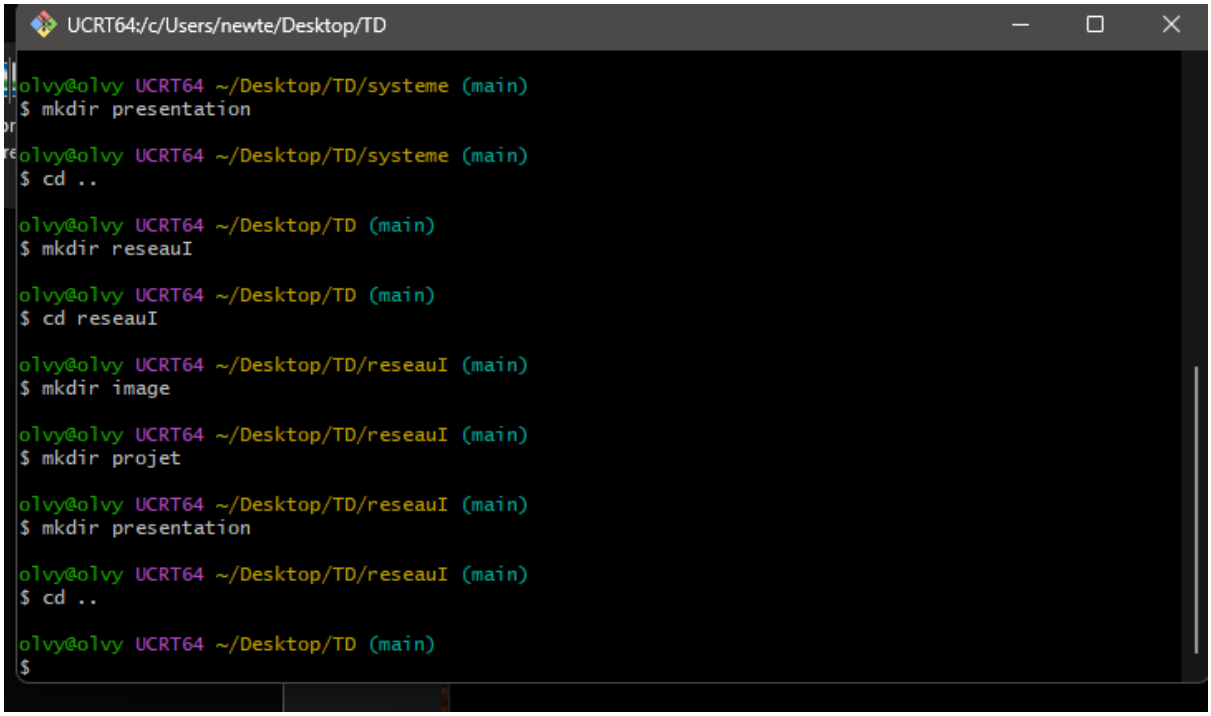
Création du dépôt public principal nommé TD sur GitHub avec une licence Apache 2.0 et un fichier README initial.



Clonage du dépôt principal TD sur le Bureau Windows avec la commande git clone git@github.com:olivierdesvallons27-sys/TD.git



Création de l'arborescence requise à l'intérieur du dépôt TD à l'aide des commandes mkdir et cd. Organisation des sous-dossiers presentation dans systeme, puis création du dossier reseaul contenant ses sous-dossiers respectifs (image, projet, presentation).



Conclusion

1. Résumé des acquis

Au cours de ce travail pratique, plusieurs compétences clés en gestion de version et administration système ont été développées et validées :

- **Installation et configuration de Git :** Prise en main du terminal (Git Bash / PowerShell) et paramétrage des identifiants globaux (user.name et user.email).
- **Gestion du cycle de vie local :** Initialisation d'un dépôt local (git init), suivi de l'état des fichiers (git status), indexation (git add), validation des modifications (git commit) et consultation de l'historique (git log).
- **Sécurisation des échanges via SSH :** Génération d'une paire de clés cryptographiques RSA (4096 bits) avec ssh-keygen, manipulation de la clé publique (id_rsa.pub) et association sécurisée au compte GitHub.
- **Synchronisation avec GitHub :** Création de dépôts distants (-Mon--Systemes-projet1 et TD), clonage local via protocole SSH (git clone), et envoi des commits vers le serveur distant (git push).
- **Organisation de projets :** Structuration propre d'une arborescence de travail complexe avec les sous-dossiers requis (systeme, reseaul, image, presentation, projet).

2. Bilan de réalisation et succès de la tâche

La tâche globale est un **succès complet**. L'ensemble de la procédure demandée pour ce TP1 a été exécuté méthodiquement :

1. Les environnements locaux et distants ont été configurés et interconnectés via SSH.
2. L'arborescence complète du dépôt principal TD a été mise en place avec tous ses sous-dossiers.

3. Le présent rapport au format PDF (Rapport1.pdf) ainsi que la totalité des captures d'écran justificatives ont été hébergés et synchronisés sur le dépôt GitHub distant.

3. Difficultés rencontrées et solutions apportées

Difficultés rencontrées	**Solutions appliquées**
Gestion et authentification SSH initiale : Compréhension de la distinction entre clé privée (id_rsa) et clé publique (id_rsa.pub) lors de la configuration sur GitHub.	Identification explicite de la clé publique via la commande cat ~/.ssh/id_rsa.pub pour ne transmettre que celle-ci à l'interface GitHub.
Avertissements sur la conversion de fin de ligne (LF / CRLF) : Messages d'avertissement affichés par Git lors de la commande git add sous Windows.	Avertissement pris en compte comme un comportement normal sous Windows sans impact critique sur l'intégrité des fichiers.
Erreurs de syntaxe légères dans le terminal : Saisie de virgules ou d'espaces inappropriés lors des commandes d'indexation (ex: git add , au lieu de git add .).	Lecture attentive des messages d'erreur du terminal et correction immédiate des commandes saisies.
Organisation rigoureuse des sous-dossiers : Risque de confusion lors de la création manuelle des nombreux dossiers (systeme/image, reseaul/projet, etc.).	Utilisation séquentielle et contrôlée des commandes mkdir et cd avec vérification systématique de l'arborescence.